

AWS Well-Architected Framework Review — ROCA Event Management Flows (Batch 2)

1 Executive Summary

The Batch 2 Well-Architected review of ROCA's Event Creation and Event Discovery/RSVP flows reveals **HIGH risk** across the Security and Reliability pillars, driven by six critical vulnerabilities that require immediate remediation. All five architectural gaps identified in the Batch 1 Onboarding review — embedded state machines, non-rotating JWT refresh tokens, missing DLQs, absent WAF/GuardDuty, and no EventBridge event bus — carry forward to the Event flows and compound in severity due to the presence of Stripe payment processing and concurrent capacity management [1, 2, 3, 4, 5, 6].

The most urgent finding is that RSVP capacity enforcement lacks atomic concurrency control, exposing the system to overbooking under simultaneous bookings. The recommended fix — PostgreSQL `SELECT FOR UPDATE SKIP LOCKED` backed by a Redis atomic `DECR` fast-path gate — serializes capacity decrements while maintaining throughput [7, 8]. Equally critical, the Stripe integration's PCI DSS posture is weakened by missing WAF protections and unverified webhook signatures, though Stripe Elements tokenization correctly minimizes cardholder data scope to SAQ A eligibility [9, 10, 11, 2]. Third, the `rrule.js` library introduces silent scheduling errors: DST transitions shift recurring event times, UNTIL serialization violates RFC 5545, and missing Luxon dependencies cause a silent fallback to UTC — a defect documented as causing wrong appointment times in production systems [12, 13, 14]. Materializing recurring instances into a PostgreSQL table with GiST-indexed `tstz` range columns resolves both parsing reliability and discovery query performance at scale [15, 16].

The remediation roadmap identifies 16 items totaling 14–18 engineering weeks. Six Critical items — WAF deployment, GuardDuty enablement, DLQ configuration, RSVP locking, webhook verification, and JWT rotation — are completable within a two-week sprint, with four requiring only one to three days of effort each. The highest-impact architectural change is migrating state transitions to AWS Step Functions with saga-pattern cancellation cascades, delivering exactly-once payment processing and built-in compensation logic for the complex refund chains triggered by event cancellations [4, 17, 18].

2 Introduction and Scope

This report presents the Batch 2 AWS Well-Architected Framework review for the ROCA church management application, covering two critical flows: **Event Creation** (a 6-step wizard supporting RRULE recurring event patterns, Stripe ticketing and payment, and volunteer role assignment) and **Event Discovery and RSVP** (a 5-step flow for browsing, searching, RSVP with capacity and waitlist management, and Stripe checkout). The technology stack under review includes AWS Lambda with Node.js Express, API Gateway REST, PostgreSQL 15 on RDS Multi-AZ, Redis 7 for caching, S3 and CloudFront for media delivery, SES and SNS for notifications, Stripe for payments, WebSocket via Socket.io for real-time updates, and JWT with OAuth 2.0 for authentication.

The Batch 1 review of the Onboarding flow identified five significant gaps: state machines embedded in application code instead of AWS Step Functions, JWT refresh tokens lacking rotation, no dead-letter queues on SQS consumers, missing WAF and GuardDuty, and no EventBridge event bus for cross-flow integration. All five of these gaps carry forward to the Event flows and are cross-referenced throughout this review. The following risk matrix summarizes the assessment across all six Well-Architected pillars for both flows.

Chart Explanation: This heatmap shows the risk severity for each Well-Architected pillar across both Event flows. Security and Reliability are rated HIGH risk for both flows due to missing WAF, GuardDuty, embedded state machines, and race condition vulnerabilities [1, 2, 3, 7, 4]. Performance Efficiency is HIGH for Event Discovery/RSVP due to RRULE parsing challenges and search scalability concerns [12, 15, 16]. Operational Excellence and Cost Optimization are MEDIUM risk, while Sustainability is LOW given the inherent efficiency of the serverless architecture [19, 20].

3 Flow Architecture and State Machine Design

Event Creation follows a 6-step wizard with states DRAFT → PUBLISHED → ACTIVE → COMPLETED, encompassing event metadata entry, RRULE recurrence configuration, Stripe ticketing setup, volunteer role assignment, media upload, and publication. **Event Discovery and RSVP** follows a 5-step flow with states PENDING → CONFIRMED → CHECKED_IN → ATTENDED, covering event browsing, search and filtering, RSVP submission with capacity checks, Stripe checkout for paid events, and attendance tracking.

Both flows currently embed state transition logic directly in application code — the same architectural gap identified in Batch 1. This approach lacks built-in audit trails, retry mechanisms, and compensation logic. AWS Step

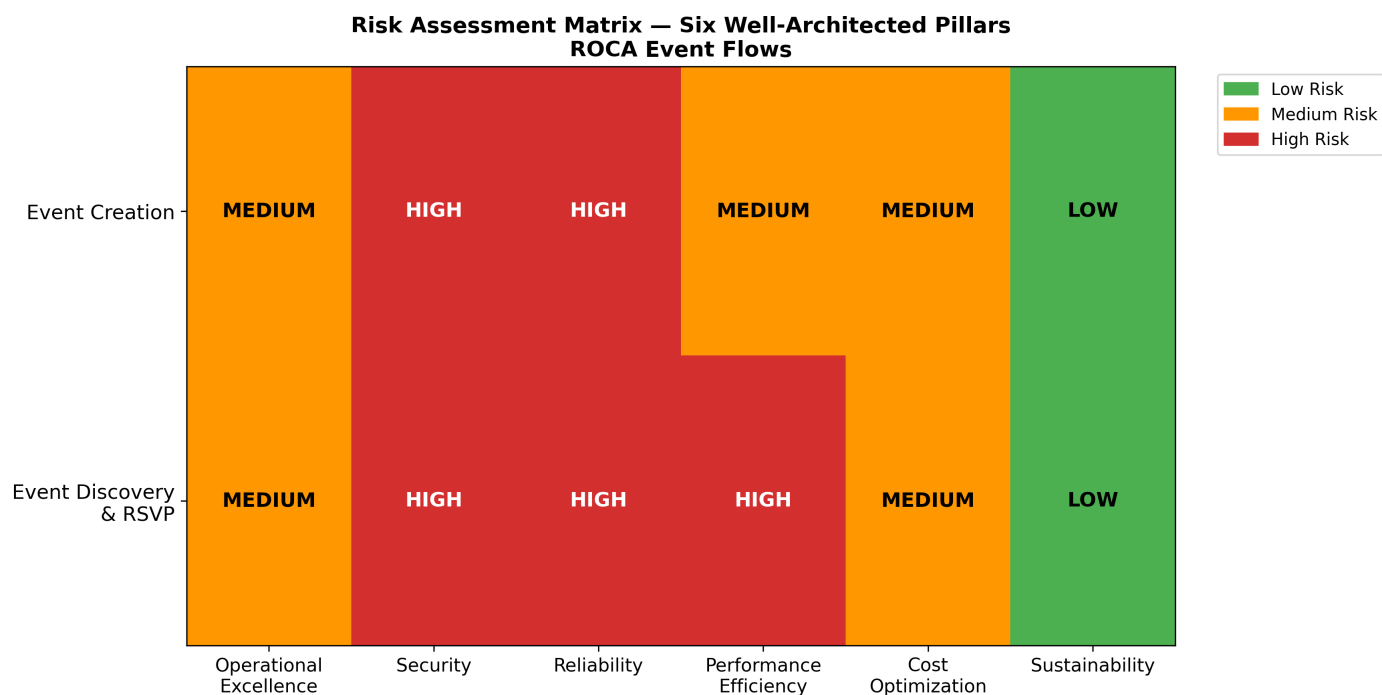


Figure 1: Risk Assessment Matrix

Functions Standard Workflows provide exactly-once execution semantics, making them suited to orchestrating non-idempotent actions such as processing payments [4]. The recommended architecture nests Express Workflows inside Standard Workflows for high-volume idempotent sub-tasks such as batch notifications, optimizing cost while maintaining execution guarantees [21].

For cross-flow integration, EventBridge should serve as the serverless event bus, decoupling producers and consumers across flows — for example, Event Creation emitting an event .cancelled event consumed by the RSVP and Notification services [22]. EventBridge supports custom retry policies per target and DLQ configuration using standard SQS queues, with CloudWatch metrics such as `InvocationsSentToDLQ` for monitoring delivery failures [5]. This architecture replaces the tightly coupled, synchronous inter-flow communication currently in place.

4 Security and PCI DSS Compliance

4.1 PCI DSS Scope Minimization

The Stripe integration architecture is the most security-sensitive component of both Event flows. Tokenization through Stripe Elements or Checkout ensures that cardholder data is collected in Stripe-hosted iframes and never touches Lambda functions or the merchant backend [10]. Systems that hold only tokens generally fall outside the scope of the full PCI DSS standard, because tokenization replaces the primary account number with a substitute value that has no exploitable value [9]. This positions ROCA for SAQ A compliance, the lightest self-assessment tier. It is important to note that PCI DSS v4.0.1 future-dated requirements became fully enforceable on March 31, 2025, and the architecture must be validated against these updated controls [23].

4.2 Webhook Security and Idempotency

Stripe webhook endpoints must verify the `Stripe-Signature` header using `constructEvent()` with the request body, signature header, and webhook signing secret to confirm events originate from Stripe [11]. TLS must be enforced for webhook endpoints to prevent traffic interception [10]. For Lambda retry safety, Stripe recommends adding idempotency keys using V4 UUIDs (up to 255 characters) to all POST requests [24]. This is especially critical for the Event Creation payment flow and RSVP checkout, where duplicate charges would directly impact church members.

4.3 AWS Security Controls

The payment architecture should route traffic through API Gateway with AWS WAF and AWS Shield protecting against application-layer exploits and DDoS attacks [1]. WAF can be directly associated with API Gateway REST APIs

to block SQL injection and XSS attacks targeting payment endpoints [2]. Stripe API secret keys must be encrypted using KMS with VPC endpoints, where endpoint policies provide additional protection [25]. Amazon GuardDuty with Lambda Protection should be enabled for detecting threats against serverless payment functions, monitoring CloudTrail events and VPC Flow Logs [3]. PCI DSS Requirement 10 mandates comprehensive CloudTrail audit logging of all access to system components and cardholder data to reconstruct events and demonstrate compliance [26].

The JWT refresh token rotation gap identified in Batch 1 remains unresolved and applies directly to Event flows. JWTs provide cryptographically secure identity verification for Lambda-based microservices [6], but without token rotation, compromised refresh tokens grant indefinite access to event management and payment endpoints.

5 Concurrency Control, Capacity Enforcement, and Overbooking Prevention

Race conditions during concurrent RSVP bookings represent one of the highest-risk areas in the Event Discovery flow. When multiple users attempt to RSVP for the last available spots simultaneously, the system must guarantee atomic capacity enforcement without overbooking.

5.1 PostgreSQL Locking Strategy

The recommended pattern is `SELECT FOR UPDATE SKIP LOCKED`, which acquires row-level locks on the event capacity row and skips already-locked rows instead of blocking, preventing convoy effects where transactions queue behind each other [7]. The transaction pattern is: begin a transaction, select the remaining capacity with `FOR UPDATE`, verify capacity is greater than zero, insert the RSVP record, decrement remaining capacity, and commit. This serializes concurrent capacity decrements atomically within PostgreSQL.

Serializable isolation is an alternative that eliminates the need for explicit locking by detecting read/write conflict cycles and aborting one transaction, but it requires application-level retry logic and has higher abort rates under contention, making it less suitable for high-demand events [27]. Advisory locks via `pg_advisory_xact_lock(event_id)` provide a lightweight alternative for coarse-grained capacity gating without touching table rows [28]. For waitlist promotion, the pattern uses `SELECT ... FROM waitlist WHERE event_id = \ $1 ORDER BY created_at FOR UPDATE SKIP LOCKED LIMIT 1` inside the cancellation transaction to atomically promote the next waitlisted entry while adjusting capacity in a single transaction.

5.2 Redis as a Fast-Path Capacity Gate

Redis atomic `DECR` operations (or Lua scripts for check-and-decrement) should serve as a fast-path capacity gate to reject requests before they hit PostgreSQL, with the database transaction serving as the authoritative source of truth [8]. If Redis indicates availability but PostgreSQL rejects the booking (capacity exhausted), the PostgreSQL transaction is authoritative — the system returns failure and asynchronously corrects the Redis counter. Redis documents the Redlock algorithm for distributed locking [29], but Martin Kleppmann's analysis cautions that Redlock is unsafe for correctness-critical use cases due to clock drift and garbage collection pause vulnerabilities — a fencing token approach is recommended instead [30]. For cache invalidation, Redis recommends cache-aside with short TTLs, write-through, or event-driven sync via CDC/streams to keep drift within acceptable bounds [31].

6 RRULE Parsing Reliability and Event Discovery Performance

6.1 Known `rrule.js` Vulnerabilities

The RRULE parsing implementation for recurring events carries several critical reliability risks. The `rrule.js` library uses an internal UTC representation for all dates, which causes recurrence series to shift time-of-day after DST transitions — for example, a 10:00 AM event in America/Denver may become 9:00 AM or 11:00 AM post-DST [12]. The root cause is that `rrule.js` internally represents timezones in UTC, so local times appear incorrect when rezoned [32]. Additionally, when `DTSTART` includes a TZID, `rrule.js` generates UNTIL without the required trailing 'Z' (UTC indicator), producing an invalid RRULE per RFC 5545 [13], and the `.after()` method breaks when TZID is present [33].

A particularly dangerous issue is that `rrule.js` requires Luxon for TZID support but silently falls back to UTC if Luxon is missing, which can cause serious business logic errors — as documented in a medical appointment system where this caused all appointments to have wrong start times [14]. Mitigations include: always passing `tzid` and post-processing generated UTC dates through Luxon's `DateTime.fromJSDate().setZone()` for correct wall-clock times; preferring `COUNT` over `UNTIL` to avoid the serialization bug; manually appending 'Z' and converting to UTC when `UNTIL` is required; and validating Luxon availability at startup with a hard failure if absent.

6.2 Materialized Instance Tables and Search Performance

For discovery queries at scale, materializing future occurrences for a bounded window (such as a rolling 12 months) into an `event_instances` table is recommended over lazy generation [15, 34]. PostgreSQL's `tstzrange` type with GiST indexes is ideal for range-containment queries using the `@>` and `&&` operators, enabling efficient date-range overlap searches [16]. A `postgres-rrule` extension can handle server-side RRULE expansion [35]. Instances should be regenerated via a scheduled job or triggered on RRULE edits.

For caching, the cache-aside pattern with query-parameter hashing (e.g., `events:{org_id}:{date_range_hash}`) as cache keys works well, storing serialized instance arrays with TTLs of 1–24 hours depending on query frequency [36]. The three main causes of cache drift — TTL windows outlasting the data, write-ordering races, and multiple app instances stepping on each other during cache fills — must be addressed through short TTLs and pub/sub invalidation on RRULE edits [31].

7 Event Cancellation Cascading Effects

Event cancellation triggers a complex chain of side effects: all RSVPs must be cancelled, paid tickets require Stripe refunds, volunteer assignments need release, and affected attendees must be notified. Partial failures in this chain can leave the system in an inconsistent state.

The saga orchestration pattern implemented via Step Functions is the recommended approach, where each step (cancel RSVPs → process Stripe refunds → send notifications) has a corresponding compensating transaction; if any step fails, Step Functions executes compensating transactions in reverse order to undo preceding changes [17]. Step Functions supports configuring automatic retries with exponential backoff and different retry policies per error type for transient failures in payment and notification steps [37]. For Stripe refund processing, Step Functions HTTP Task can natively call third-party HTTPS APIs like Stripe directly from a Task state without requiring a Lambda intermediary [18]. For asynchronous Stripe operations such as large batch refunds, the callback task token pattern allows Step Functions to pause until the external system returns the token [38].

For fan-out notifications, SNS serves as an EventBridge rule target to fan out to multiple subscribers — SES for email and SNS for push/SMS — with SNS supporting its own DLQ when delivery policies are exhausted [39]. Standard Workflows' exactly-once execution model combined with Stripe idempotency keys prevents duplicate refunds during retries [40, 41].

8 Operational Excellence, Performance, Cost, and Sustainability

8.1 Operational Excellence

Lambda Powertools for TypeScript provides structured logging, X-Ray tracing, and custom CloudWatch metrics as a unified toolkit for serverless observability [42]. X-Ray distributed tracing integrates with Lambda and API Gateway to trace requests across services, with AWS recommending ADOT, X-Ray SDK, or Powertools as tracing libraries [43]. The circuit breaker pattern should be implemented using Lambda extensions with DynamoDB to track failure states, preventing cascading failures to downstream services like RDS and Stripe [44]. For Lambda functions in VPCs, NAT gateways should be configured across multiple Availability Zones for high availability [45].

8.2 Performance Efficiency

Provisioned Concurrency maintains pre-initialized execution environments for double-digit millisecond response times, eliminating cold starts for Node.js Lambda functions [46]. SnapStart currently supports Java, Python, and .NET but not Node.js as of mid-2025 [47]. For Node.js, AWS recommends bundling with SDK v3 and using ES modules to reduce cold start times [48]. RDS Proxy provides connection pooling that significantly reduces connection latency and enables higher database throughput, which is critical for Lambda-to-PostgreSQL workloads with high connection churn during peak RSVP periods [49].

8.3 Cost Optimization and Sustainability

Compute Savings Plans automatically reduce costs on Lambda, EC2, and Fargate usage with savings up to 72% off On-Demand pricing, offering more flexibility than Reserved Instances for serverless workloads [50]. RDS Reserved Instances should be used for the predictable Multi-AZ PostgreSQL deployment, and ElastiCache Reserved Nodes for Redis. According to the AWS Sustainability Pillar, serverless architectures inherently improve sustainability by provisioning resources only when needed, and caching via CloudFront and ElastiCache reduces redundant compute and origin requests [19]. AWS also recommends adopting serverless architectures for data pipelines to avoid maintaining compute infrastructure continuously [20].

9 Cross-Reference: Batch 1 Onboarding Gaps Applied to Event Flows

All five gaps identified in the Batch 1 Onboarding review apply directly to the Event Creation and Event Discovery/RSVP flows. The following table summarizes each gap, its Event flow impact, and the compounding risk of leaving it unresolved.

Batch 1 Gap	Event Flow Impact	Compounding Risk
Embedded state machine in application code	Event Creation (DRAFT→COMPLETED) and RSVP (PENDING→ATTENDED) lack audit trails, retry logic, and compensation patterns [4, 21]	Payment state transitions without exactly-once guarantees risk duplicate charges or lost refunds
JWT refresh tokens lacking rotation	Compromised tokens grant indefinite access to event management and payment endpoints [6]	Combined with missing WAF, attackers can exploit payment APIs with stolen tokens
No dead-letter queues on SQS consumers	Failed RSVP confirmations, refund notifications, and capacity updates are silently lost [5]	Cancellation cascades fail silently, leaving orphaned RSVPs and unprocessed refunds
Missing WAF and GuardDuty	Payment endpoints exposed to SQL injection, XSS, and undetected threats [2, 3]	PCI DSS non-compliance; Stripe integration security posture degraded
No EventBridge event bus for cross-flow integration	Event Creation cannot emit lifecycle events consumed by RSVP, Notification, and Volunteer services [22]	Tight coupling forces synchronous calls, increasing latency and failure blast radius

Table: Batch 1 Onboarding Gaps — Applicability to Event Flows

The compounding nature of these gaps is significant. For example, the combination of missing WAF, absent GuardDuty, and non-rotating JWT tokens creates a layered vulnerability where an attacker could exploit an unprotected API Gateway endpoint, use a stolen refresh token for persistent access, and trigger fraudulent payment operations — all without detection.

10 Prioritized Remediation Roadmap

The following chart and table present the prioritized remediation recommendations organized by priority tier and effort estimate.

Chart Explanation: This chart visualizes all 16 remediation items ordered by priority (Critical in red, High in orange, Medium in blue) with bar width indicating relative implementation effort. Six Critical items should be addressed in the first sprint, with WAF deployment, GuardDuty enablement, and DLQ configuration being quick wins requiring low effort [1, 2, 3, 5].

Priority	Remediation Item	Effort	Timeline	Key Citations
Critical	Deploy AWS WAF on API Gateway REST endpoints	Low (1–2 days)	Sprint 1	[1, 2]
Critical	Enable GuardDuty with Lambda Protection	Low (1 day)	Sprint 1	[3]
Critical	Add DLQs to all SQS consumers and EventBridge targets	Low (2–3 days)	Sprint 1	[5]
Critical	Implement SELECT FOR UPDATE SKIP LOCKED for RSVP capacity	Medium (1 week)	Sprint 1	[7]

Priority	Remediation Item	Effort	Timeline	Key Citations
Critical	Enforce Stripe webhook signature verification	Low (1–2 days)	Sprint 1	[11, 10]
Critical	Implement JWT refresh token rotation	Medium (1 week)	Sprint 1	[6]
High	Migrate state machines to Step Functions Standard Workflows	High (3–4 weeks)	Sprint 2–3	[4, 21, 17]
High	Implement EventBridge event bus for cross-flow integration	Medium (2 weeks)	Sprint 2	[22, 5]
High	Build saga pattern for cancellation cascades	High (3–4 weeks)	Sprint 3–4	[17, 37, 18]
High	Materialize RRULE instances into PostgreSQL with GiST indexes	Medium (2 weeks)	Sprint 2	[15, 34, 16]
High	Fix rrule.js DST drift and UNTIL serialization bugs	Medium (1–2 weeks)	Sprint 2	[12, 32, 13, 14]
High	Implement Redis atomic DECR capacity gate	Medium (1 week)	Sprint 2	[8, 30]
Medium	Deploy RDS Proxy for Lambda connection pooling	Low (2–3 days)	Sprint 3	[49]
Medium	Configure Provisioned Concurrency for critical Lambda functions	Low (1–2 days)	Sprint 3	[46, 48]
Medium	Adopt Compute Savings Plans and Reserved Instances	Low (1 day)	Sprint 3	[50]
Medium	Integrate Lambda Powertools with X-Ray distributed tracing	Medium (1–2 weeks)	Sprint 3	[42, 43]

Table: Prioritized Remediation Recommendations with Effort Estimates

Sprint 1 (Weeks 1–2) focuses on six Critical items, four of which are low-effort quick wins. WAF deployment, GuardDuty enablement, DLQ configuration, and Stripe webhook verification can be completed in under a week combined. The SELECT FOR UPDATE SKIP LOCKED implementation and JWT rotation require approximately one week each but address the highest-severity risks: overbooking and authentication bypass.

Sprint 2–3 (Weeks 3–8) addresses High-priority architectural improvements. The Step Functions migration is the largest single effort at 3–4 weeks but delivers the most transformative improvement — exactly-once payment processing, built-in saga compensation, and native audit trails. EventBridge integration and RRULE materialization can proceed in parallel. The rrule.js bug fixes should be prioritized early in Sprint 2 to prevent DST-related scheduling errors for recurring church events.

Sprint 3–4 (Weeks 6–10) completes Medium-priority optimizations. RDS Proxy, Provisioned Concurrency, and Savings Plans are low-effort items that deliver measurable performance and cost improvements. Lambda Power-tools integration provides the observability foundation needed to monitor all preceding changes in production.

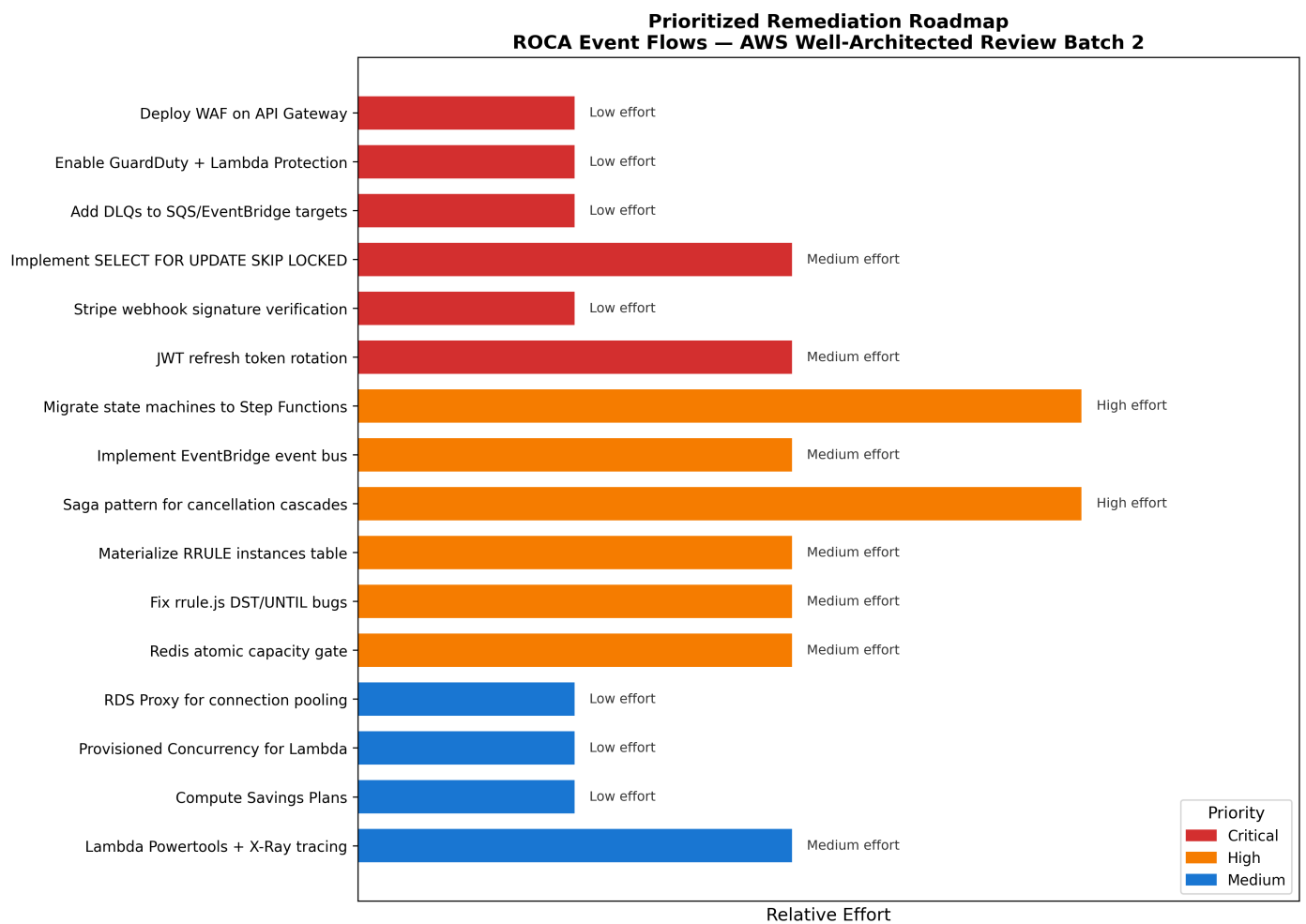


Figure 2: Prioritized Remediation Roadmap

The total estimated effort across all 16 items is approximately 14–18 engineering weeks, with the Critical tier completable in 2 weeks and delivering immediate risk reduction across Security, Reliability, and PCI DSS compliance.

11 References

1. <https://docs.aws.amazon.com/wellarchitected/latest/financial-services-industry-lens/payments.html>
2. <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-control-access-aws-waf.html>
3. https://docs.aws.amazon.com/guardduty/latest/ug/guardduty_settingup.html
4. <https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html>
5. <https://docs.aws.amazon.com/eventbridge/latest/userguide/rule-dlq.html>
6. <https://aws.amazon.com/blogs/compute/operating-lambda-building-a-solid-security-foundation-part-2/>
7. <https://www.cybertec-postgresql.com/en/select-for-update-considered-harmful-postgresql/>
8. <https://redis.io/tutorials/inventory-reservation-in-real-time-with-redis/>
9. <https://stripe.com/resources/more/pci-tokenization>
10. <https://docs.stripe.com/security/guide>
11. <https://docs.stripe.com/webhooks/signature>
12. <https://github.com/jkbrzt/rrule/issues/550>
13. <https://github.com/jakubroztocil/rrule/issues/440>
14. <https://github.com/jakubroztocil/rrule/issues/427>
15. <https://thoughtbot.com/blog/recurring-events-and-postgresql>
16. <https://www.postgresql.org/docs/current/rangetypes.html>
17. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/implement-the-serverless-saga-pattern-by-using-aws-step-functions.html>
18. <https://docs.aws.amazon.com/step-functions/latest/dg/integrate-services.html>
19. <https://docs.aws.amazon.com/wellarchitected/latest/sustainability-pillar/sustainability-pillar.html>
20. <https://docs.aws.amazon.com/solutions/optimizing-deep-learning-workloads-for-sustainability-on-aws/>
21. <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html>
22. <https://docs.aws.amazon.com/prescriptive-guidance/latest/patterns/create-cross-account-amazon-eventbridge-connection-organization.html>
23. <https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-security-reference-architecture-pci-data-security-standard-dss/introduction.html>
24. https://docs.stripe.com/api/idempotent_requests?api-version=2024-06-20
25. <https://docs.aws.amazon.com/solutions/tokenization-to-improve-data-security-and-reduce-audit-scope-on-aws/>
26. <https://docs.aws.amazon.com/prescriptive-guidance/latest/aws-security-reference-architecture-payment-card-industry-pci-data-security-standard-dss/logging-monitoring-and-incident-response.html>
27. <https://www.postgresql.org/docs/current/applevel-consistency.html>
28. <https://www.postgresql.org/docs/13/explicit-locking.html>
29. <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>
30. <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>
31. <https://redis.io/blog/cache-consistency-strategies/>
32. <https://github.com/jakubroztocil/rrule/issues/253>
33. <https://github.com/jkbrzt/rrule/issues/608>
34. https://codemia.io/knowledge-hub/path/in_a_database_how_to_store_event_occurrence_dates_and_timeframes_f

35. <https://github.com/volkanunsal/postgres-rrule>
36. <https://redis.io/tutorials/howtos/solutions/microservices/caching/>
37. <https://docs.aws.amazon.com/lambda/latest/dg/with-step-functions.html>
38. <https://docs.aws.amazon.com/step-functions/latest/dg/connect-to-resource.html>
39. <https://aws.amazon.com/ko/blogs/compute/choosing-between-messaging-services-for-serverless-applications/>
40. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/step-functions-workflows.html>
41. https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/rel_prevent_interaction_failure_idempotent.html
42. <https://docs.aws.amazon.com/powertools/typescript/main/llms-full.txt>
43. <https://docs.aws.amazon.com/lambda/latest/dg/typescript-tracing.html>
44. <https://docs.aws.amazon.com/prescriptive-guidance/latest/cloud-design-patterns/circuit-breaker.html>
45. <https://docs.aws.amazon.com/wellarchitected/latest/serverless-applications-lens/aws-lambda.html>
46. <https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>
47. <https://aws.amazon.com/about-aws/whats-new/2025/06/aws-lambda-snapstart-python-net-functions-23-regions/>
48. <https://aws.amazon.com/fr/blogs/developer/reduce-lambda-cold-start-times-migrate-to-aws-sdk-for-javascript-v3/>
49. <https://docs.aws.amazon.com/prescriptive-guidance/latest/amazon-rds-proxy>
50. <https://docs.aws.amazon.com/savingsplans/latest/userguide/sp-ris.html>